

Librerie Python da conoscere per probabilità e statistica

In Python esistono diverse librerie che permettono di lavorare con distribuzioni di probabilità, simulazioni, statistiche descrittive, inferenza e modelli statistici.

In particolare segnaliamo:

- `scipy.stats`
- `random`
- `statsmodels`

E' utile andare a guardare i siti di riferimento dove sono presenti le spiegazioni dettagliate delle varie funzioni e dei tutorial.

Per esempio: <https://docs.scipy.org/doc/scipy/reference/stats.html>

Parleremo anche di altre librerie utilizzate nella pratica statistica e nel data science.

1. `scipy.stats`

`scipy.stats` è la libreria più completa e avanzata di Python per lavorare con:

- distribuzioni di probabilità (discrete e continue)
- test statistici (vedremo in futuro)
- stime dei parametri (vedremo in futuro)
- funzioni di densità e distribuzione
- generazione di campioni da distribuzioni arbitrarie

Funzionalità principali

1. Distribuzioni di probabilità

La libreria contiene decine di distribuzioni, ad esempio:

```
from scipy.stats import norm, binom, poisson, uniform
```

Per ogni distribuzione sono disponibili:

- **PDF** (probability density function): `norm.pdf(x)`
- **CDF** (cumulative distribution function): `norm.cdf(x)`
- **PPF** (inverse CDF): `norm.ppf(q)`
- **Generazione campioni**: `norm.rvs(size=1000)`

Esempio:

```
from scipy.stats import norm

x = norm.rvs(loc=0, scale=1, size=1000)
```

rvs significa *random variates*: genera valori casuali estratti da una distribuzione specifica.

2. Test statistici (Vedremo in futuro)

Alcuni test disponibili:

- t-test: `ttest_ind`, `ttest_rel`
- chi-quadro: `chisquare`
- Kolmogorov-Smirnov: `ks_2samp`
- Shapiro-Wilk: `shapiro`
- ANOVA: `f_oneway`

Esempio:

```
from scipy.stats import ttest_ind
ttest_ind(x1, x2)
```

3. Stima dei parametri

Molte distribuzioni permettono il fitting:

```
params = norm.fit(data) # stima mu, sigma
```

2. `random` (libreria standard Python)

La libreria `random` è molto semplice e serve per:

- generare numeri casuali
- simulare eventi
- estrarre campioni da liste o intervalli

Non è pensata per la statistica avanzata, ma è utile per coding di base o simulazioni semplici.

Esempi di utilizzo

Generare numeri casuali da uniformi

```
import random
x = random.random() # uniforme tra 0 e 1
```

Interi casuali

```
n = random.randint(1, 6) # simulazione di un dado
```

Campionamento da liste

```
random.choice(['Testa', 'Croce'])
```

Campionamento senza rimpiazzo

```
random.sample([1,2,3,4,5], k=2)
```

È utile per simulare eventi come estrazioni, lancio di dadi, monete, ecc.

3. statsmodels

`statsmodels` è la libreria di riferimento per la **statistica classica** e per una parte dell'econometria.

Permette di fare:

- regressioni lineari con analisi dei residui
- modelli statistici classici (OLS, GLM, ANOVA, ARIMA)
- test statistici avanzati
- stima dei parametri e inferenza
- analisi delle serie temporali

Esempio di regressione lineare

```
import statsmodels.api as sm

X = sm.add_constant(df[['x1', 'x2']])
y = df['y']

model = sm.OLS(y, X).fit()
print(model.summary())
```

La colonna `y` del DataFrame è la variabile che vogliamo predire.

`df[['x1', 'x2']]` seleziona due colonne del DataFrame come variabili indipendenti (predictors).

`sm.add_constant(...)` aggiunge una colonna di 1 che serve per stimare l'intercetta (β_0) del modello lineare.

In statsmodels, se non la aggiungi tu, non viene aggiunta automaticamente (diversamente da scikit-learn).

Ulterio approfondimenti:

- OLS = Ordinary Least Squares, regressione lineare ai minimi quadrati.
- OLS(y, X) costruisce il modello.
- .fit() lo addestra (stima i coefficienti β).

Dopo .fit() si ha un modello addestrato con:

- coefficienti,
- intervalli di confidenza,
- p-values,
- statistiche diagnostiche.

`statsmodels` fornisce risultati dettagliati qui inclusi:

- stime dei coefficienti
- p-value
- intervalli di confidenza
- R^2 e statistiche diagnostiche
- test di normalità dei residui

4. Altre librerie utili per probabilità e statistica

4.1 `numpy`

Non nasce per la statistica, ma è fondamentale per:

- vettorizzazione
- calcolo di media, varianza, correlazioni
- generazione di numeri casuali (modulo `numpy.random`)

Esempio:

```
import numpy as np
x = np.random.normal(0, 1, 1000)
```

4.2 `pandas`

Utile per:

- operazioni sui dataset
- statistiche descrittive
- manipolazione e pulizia dei dati

Esempi:

```
df.describe()
df['col'].mean()
df['col'].var()
```

4.3 matplotlib e seaborn

Per visualizzare distribuzioni, istogrammi, boxplot, density plot, ecc.

Esempio:

```
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt

# 1. Creo una distribuzione normale simulata
data = np.random.normal(loc=0, scale=1, size=1000)

# 2. Creo un grafico semplice con histogram + KDE
sns.histplot(data, kde=True)

plt.title("Distribuzione normale simulata")
plt.xlabel("Valore")
plt.ylabel("Frequenza / Densità")
plt.show()
```

4.4 (CENNI) PyMC o PyMC3

Per la **statistica bayesiana** e modelli probabilistici complessi:

- definizione di modelli gerarchici
- inferenza tramite MCMC
- posteriori e predittivi

4.5 sklearn

Per modelli predittivi che usano concetti probabilistici:

- stime di densità (Kernel Density Estimation)
- Gaussian Mixture Models
- Naive Bayes
- MA SOPRATTUTTO: una delle principali libreria per il Machine Learning!

Inizia a programmare o [genera](#) codice con l'IA.

